

习题六

一、选择题

- 已知一算术表达式的中缀形式为 $A+B \times C-D/E$, 后缀形式为 $ABC++DE/-$, 其前缀形式为()
A. $-A+B \times C/DE$ B. $-A+B \times CD/E$ C. $-+ \times ABC/DE$ D. $-+A \times BC/DE$
- 设树T的度为4, 其中度为1, 2, 3和4的结点个数分别为4, 2, 1, 1 则T中的叶子数为()
A. 5 B. 6 C. 7 D. 8
- 在下述结论中, 正确的是()
①只有一个结点的二叉树的度为0; ②二叉树的度为2;
③二叉树的左右子树可任意交换;
④深度为K的完全二叉树的结点个数小于或等于深度相同的满二叉树。
A. ①②③ B. ②③④ C. ②④ D. ①④
- 设森林F对应的二叉树为B, 它有m个结点, B的根为p, p的右子树结点数为n, 森林F中第一棵树的结点个数是()
A. $m-n$ B. $m-n-1$ C. $n+1$ D. 条件不足, 无法确定
- 若一棵二叉树具有10个度为2的结点, 5个度为1的结点, 则度为0的结点个数是()
A. 9 B. 11 C. 15 D. 不确定
- 一棵完全二叉树上有1001个结点, 其中叶子结点的个数是()
A. 250 B. 500 C. 254 D. 505 E. 以上答案都不对
- 有n个叶子的哈夫曼树的结点总数为()。
A. 不确定 B. $2n$ C. $2n+1$ D. $2n-1$
- 二叉树的第i层上最多含有结点数为()
A. 2^i B. $2^{i-1}-1$ C. 2^{i-1} D. 2^i-1
- 一个具有1025个结点的二叉树的高h为()
A. 11 B. 10 C. 11至1025之间 D. 10至1024之间
- 一棵二叉树高度为h, 所有结点的度或为0, 或为2, 则这棵二叉树最少有()结点
A. $2h$ B. $2h-1$ C. $2h+1$ D. $h+1$
- 对于有n个结点的二叉树, 其高度为()
A. $n \log_2 n$ B. $\log_2 n$ C. $\lfloor \log_2 n \rfloor + 1$ D. 不确定
- 一棵具有n个结点的完全二叉树的树高度(深度)是()
A. $\lfloor \log n \rfloor + 1$ B. $\log n + 1$ C. $\lfloor \log n \rfloor$ D. $\log n - 1$
- 深度为h的满m叉树的第k层有()个结点。(1= k ≤ h)
A. m^{k-1} B. m^k-1 C. m^{h-1} D. m^h-1
- 将有关二叉树的概念推广到三叉树, 则一棵有244个结点的完全三叉树的高度()
A. 4 B. 5 C. 6 D. 7
- 利用二叉链表存储树, 则根结点的右指针是()。
A. 指向最左孩子 B. 指向最右孩子 C. 空 D. 非空
- 对二叉树的结点从1开始进行连续编号, 要求每个结点的编号大于其左、右孩子的编号, 同一结点的左右孩子中, 其左孩子的编号小于其右孩子的编号, 可采用()次序的遍历实现编号。
A. 先序 B. 中序 C. 后序 D. 从根开始按层次遍历
- 树的后根遍历序列等同于该树对应的二叉树的()。
A. 先序序列 B. 中序序列 C. 后序序列
- 若二叉树采用二叉链表存储结构, 要交换其所有分支结点左右子树的位置, 利用()遍历方法最合适。
A. 前序 B. 中序 C. 后序 D. 按层次
- 在下列存储形式中, 哪一个不是树的存储形式?()
A. 双亲表示法 B. 孩子链表表示法 C. 孩子兄弟表示法 D. 顺序存储表示法
- 某二叉树T有n个结点, 设按某种顺序对T中的每个结点进行编号, 编号为1, 2, ..., n, 且有如下性质: T

中任一结点 V，其编号等于左子树上的最小编号减 1，而 V 的右子树的结点中，其最小编号等于 V 左子树上结点的最大编号加 1。这时是按()编号的。

A.中序遍历序列 B.前序遍历序列 C.后序遍历序列 D.层次顺序

21. 下面的说法中正确的是 ()。

(1) 任何一棵二叉树的叶子结点在三种遍历中的相对次序不变；

(2) 按二叉树定义，具有三个结点的二叉树共有 6 种。

A . (1)(2) B . (1) C . (2) D . (1)、(2)都错

22. 某二叉树的前序序列和后序序列正好相反，则该二叉树一定是 () 的二叉树。

A . 空或只有一个结点 B . 任一结点无左子树

C . 高度等于其结点数 D . 任一结点无右子树

23. 一棵左子树为空的二叉树在先序线索化后，其中空的链域的个数是：()

A . 不确定 B. 0 C. 1 D. 2

24. 一棵左右子树均不空的二叉树在先序线索化后，其中空的链域的个数是：()。

A. 0 B. 1 C. 2 D. 不确定

25. 若 X 是二叉中序线索树中一个有左孩子的结点，且 X 不为根，则 x 的前驱为()

A.X 的双亲 B.X 的右子树中最左的结点

C.X 的左子树中最右结点 D.X 的左子树中最右叶结点

26. 引入二叉线索树的目的是 ()

A . 加快查找结点的前驱或后继的速度 B . 为了能在二叉树中方便地进行插入与删除

C . 为了能方便的找到双亲 D . 使二叉树的遍历结果唯一

27. 线索二叉树是一种 () 结构。

A . 逻辑 B . 逻辑和存储 C . 物理 D . 线性

28. n 个结点的线索二叉树上含有的线索数为 ()

A . $2n$ B . $n-1$ C . $n+1$ D . n

29. 设 F 是一个森林，B 是由 F 变换得的二叉树。若 F 中有 n 个非终端结点，则 B 中右指针域为空的结点有 () 个。

A . $n-1$ B . n C . $n+1$ D . $n+2$

30. 由 3 个结点可以构造出多少种不同的二叉树？()

A . 2 B . 3 C . 4 D . 5

31. 下述编码中哪一个不是前缀码 ()。

A . (00, 01, 10, 11) B . (0, 1, 00, 11) C . (0, 10, 110, 111) D . (1, 01, 000, 001)

32. 当一棵有 n 个结点的二叉树按层次从上到下，同层次从左到右将数据存放在一维数组 A[1..n] 中时，数组中第 i 个结点的左孩子为 ()

A . $A[2i](2i \leq n)$ B. $A[2i+1](2i+1 \leq n)$ C . $A[i/2]$ D . 无法确定

二、判断题

1. 二叉树是度为 2 的有序树。

2. 完全二叉树一定存在度为 1 的结点。

3. 对于有 N 个结点的二叉树，其高度为 $\log_2 n$ 。

4. 深度为 K 的二叉树中结点总数 $\leq 2^k - 1$ 。

5. 二叉树的遍历结果不是唯一的。

6. 二叉树的遍历只是为了在应用中找到一种线性次序。

7. 一个树的叶结点，在前序遍历和后序遍历下，皆以相同的相对位置出现。

8. 二叉树的前序遍历并不能唯一确定这棵树，但是，如果我们还知道该树的根结点是那一个，则可以确定这棵二叉树。

9. 一棵一般树的结点的前序遍历和后序遍历分别与它相应二叉树的结点前序遍历和后序遍历是一致的。

10. 对一棵二叉树进行层次遍历时，应借助于一个栈。

11. 用树的前序遍历和中序遍历可以导出树的后序遍历。

12. 采用二叉链表作存储结构, 树的前序遍历和其相应的二叉树的前序遍历的结果是一样的。
13. 中序遍历一棵二叉排序树的结点就可得到排好序的结点序列
14. 后序线索二叉树是不完善的, 要对它进行遍历, 还需要使用栈。
15. 任何一棵二叉树都可以不用栈实现前序线索树的前序遍历。
16. 由一棵二叉树的前序序列和后序序列可以唯一确定它。
17. 完全二叉树中, 若一个结点没有左孩子, 则它必是树叶。
18. 二叉树只能用二叉链表表示。
19. 一棵有 n 个结点的二叉树, 从上到下, 从左到右用自然数依次给予编号, 则编号为 i 的结点的左儿子的编号为 $2i(2i < n)$, 右儿子是 $2i+1(2i+1 < n)$ 。
20. 给定一棵树, 可以找到唯一的一棵二叉树与之对应。
21. 一棵树中的叶子数一定等于与其对应的二叉树的叶子数。
22. 用链表(link-rlink)存储包含 n 个结点的二叉树, 结点的 $2n$ 个指针区域中有 $n-1$ 个空指针。
23. 树形结构中元素之间存在一个对多个的关系。
24. 在二叉树的第 i 层上至少有 2^{i-1} 个结点($i \geq 1$)。
25. 必须把一般树转换成二叉树后才能进行存储。
26. 完全二叉树的存储结构通常采用顺序存储结构。
27. 将一棵树转成二叉树, 根结点没有左子树;
28. 在二叉树中插入结点, 则此二叉树便不再是二叉树了。
29. 非空的二叉树一定满足:某结点若有左孩子, 则其中序前驱一定没有右孩子
30. 在任意一棵非空二叉排序树, 删除某结点后又将其插入, 则所得二叉排序树与删除前原二叉排序树相同。
31. 度为二的树就是二叉树。
32. 深度为 k 具有 n 个结点的完全二叉树, 其编号最小的结点序号为 $\lfloor 2^{k-2} \rfloor + 1$ 。
33. 线索二叉树的优点是便是在中序下查找前驱结点和后继结点。
34. 霍夫曼树的结点个数不能是偶数。
35. 一棵哈夫曼树的带权路径长度等于其中所有分支结点的权值之和。
36. 哈夫曼树无左右子树之分。
37. 当一棵具有 n 个叶子结点的二叉树的 WPL 值为最小时, 称其树为 Huffman 树, 且其二叉树的形状必是唯一的。
38. 哈夫曼树是带权路径长度最短的树, 路径上权值较大的结点离根较近。

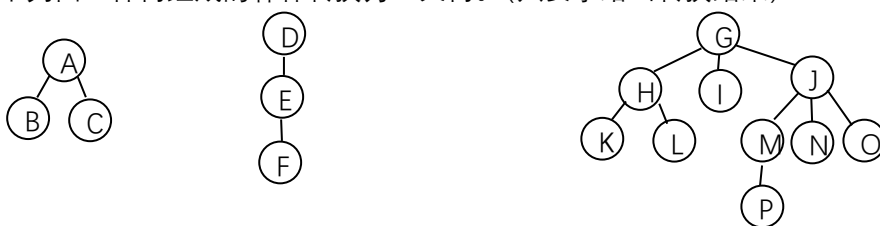
三、填空题

1. 二叉树由 (1) , (2) , (3) 三个基本单元组成。
2. 在二叉树中, 指针 p 所指结点为叶子结点的条件是_____。
3. 中缀式 $a+b*3+4*(c-d)$ 对应的前缀式为 (1) , 若 $a=1, b=2, c=3, d=4$, 则后缀式 $db/cc*a-b**$ 的运算结果为 (2) 。
4. 二叉树中某一结点左子树的深度减去右子树的深度称为该结点的_____。
5. 具有 256 个结点的完全二叉树的深度为_____。
6. 已知一棵度为 3 的树有 2 个度为 1 的结点, 3 个度为 2 的结点, 4 个度为 3 的结点, 则该树有_____个叶子结点。
7. 深度为 H 的完全二叉树至少有 (1) 个结点; 至多有 (2) 个结点; H 和结点总数 N 之间的关系是 (3) 。
8. 在完全二叉树中, 编号为 i 和 j 的两个结点处于同一层的条件是_____。
9. 一棵有 n 个结点的满二叉树有 (1) 个度为 1 的结点、有 (2) 个分支 (非 终端) 结点和 (3) 个叶子, 该满二叉树的深度为 (4) 。
10. 在一棵二叉树中, 度为零的结点的个数为 N_0 , 度为 2 的结点的个数为 N_2 , 则有 $N_0 =$ _____
11. 设有 N 个结点的完全二叉树顺序存放在向量 $A[1:N]$ 中, 其下标值最大的分支结点为_____。
12. 高度为 K 的完全二叉树至少有_____个叶子结点。
13. 已知二叉树有 50 个叶子结点, 则该二叉树的总结点数至少是_____。
14. 如某二叉树有 20 个叶子结点, 有 30 个结点仅有一个孩子, 则该二叉树的总结点数为_____。
15. n (n 大于 1) 个结点的各棵树中, 其深度最小的那棵树的深度是 (1) 。它共有 (2) 个叶子结点和 (3) 个非叶子结点, 其中深度最大的那棵树的深度是 (4) , 它共有 (5) 个叶子结点和 (6) 个非叶子结点。

16. 已知一棵二叉树的前序序列为 abdecfhg,中序序列为 dbeahfcg, 则该二叉树的根为(1), 左子树中有(2), 右子树中有(3)。
17. 设二叉树中每个结点均用一个字母表示, 若一个结点的左子树或右子树为空, 用 . 表示, 现前序遍历二叉树, 访问的结点的序列为 ABD.G...CE.H..F ., 则中序遍历二叉树时, 访问的结点序列为(1);后序遍历二叉树时, 访问的结点序列为(2)。
18. 一个无序序列可以通过构造一棵_____树而变成一个有序序列, 构造树的过程即为对无序序列进行排序的过程。
19. 利用树的孩子兄弟表示法存储, 可以将一棵树转换为_____。
20. 若一个二叉树的叶子结点是某子树的中序遍历序列中的最后一个结点, 则它必是该子树的_____序列中的最后一个结点。
21. 在一棵存储结构为三叉链表的二叉树中, 若有一个结点是它的双亲的左子女, 且它的双亲有右子女, 则这个结点在后序遍历中的后继结点是_____。
22. 设一棵后序线索树的高是 50, 结点 x 是树中的一个结点, 其双亲是结点 y,y 的右子树高度是 31, x 是 y 的左孩子。则确定 x 的后继最多需经过_____中间结点 (不含后继及 x 本身)
23. 有一份电文中共使用 6 个字符:a,b,c,d,e,f,它们的出现频率依次为 2,3,4,7,8,9, 试构造一棵哈夫曼树, 则其加权路径长度 WPL 为(1), 字符 c 的编码是(2)。
24. 设 n_0 为哈夫曼树的叶子结点数, 则该哈夫曼树共有_____个结点。

四、应用题

1. 一个深度为 L 的满 K 叉树有以下性质: 第 L 层上的结点都是叶子结点, 其余各层上每个结点都有 K 棵非空子树, 如果按层次顺序从 1 开始对全部结点进行编号, 求:
- 1) 各层的结点的数目是多少?
 - 2) 编号为 n 的结点的双亲结点 (若存在) 的编号是多少?
 - 3) 编号为 n 的结点的第 i 个孩子结点 (若存在) 的编号是多少?
 - 4) 编号为 n 的结点有右兄弟的条件是什么? 如果有, 其右兄弟的编号是多少?
- 请给出计算和推导过程。
2. 对于任何一棵非空的二叉树,假设叶子结点的个数为 n_0 ,而次数为 2 的结点个数为 n_2 ,请给出 n_0 和 n_2 之间所满足的关系式 $n_0=f(n_2)$.要求给出推导过程。
3. 如果给出了一个二叉树结点的前序序列和对称序序列, 能否构造出此二叉树?若能, 请证明之。若不能, 请给出反例。如果给出了一个二叉树结点的前序序列和后序序列, 能否构造出此二叉树?若能, 请证明之。若不能, 请给出反例。
4. 将下列由三棵树组成的森林转换为二叉树。(只要求给出转换结果)



5. 设一棵二叉树的先序、中序遍历序列分别为
 先序遍历序列: A B D F C E G H 中序遍历序列: B F D A G E H C
 (1) 画出这棵二叉树。
 (2) 画出这棵二叉树的后序线索树。
 (3) 将这棵二叉树转换成对应的树 (或森林)。
6. 一棵二叉树的先序、中序、后序序列如下, 其中一部分未标出, 请构造出该二叉树。
 先序序列: _ _ C D E _ G H I _ K
 中序序列: C B _ _ F A _ J K I G
 后序序列: _ E F D B _ J I H _ A

7. 将下列树的孩子—兄弟链表改为后根遍历全线索链表。

Data	A	B	C	D	E	F	G	H	I	J	K
Ltag	0	0	0	0	0	0	0	0	0	0	0
Fch	2	0	5	7	8	0	11	0	0	0	0
Rtag	0	0	0	0	0	0	0	0	0	0	0
Nsib	0	3	4	0	6	0	0	9	10	0	0

8. 给定权 $W_1, W_2, \dots, W_m$ 。说明怎样来构造一个具有最小的加权路径长度的 k 叉树。试对于权 1,4,9,16,25, 36,49,64,81,100 来构造最优的三叉树，并给出其最小加权路径长度。

9. 已知下列字符 A、B、C、D、E、F、G 的权值分别为 3、12、7、4、2、8、11，试填写出其对应哈夫曼树 HT 的存储结构的初态和终态。

五、算法设计题

- 编程求以孩子—兄弟表示法存储的森林的叶子结点数。要求描述结构。
- 假定用两个一维数组 $L[N]$ 和 $R[N]$ 作为有 N 个结点 $1, 2, \dots, N$ 的二元树的存储结构。 $L[i]$ 和 $R[i]$ 分别指示结点 i 的左儿子和右儿子； $L[i]=0$ ($R[i]=0$) 表示 i 的左 (右) 儿子为空。试写一个算法，由 L 和 R 建立一个一维数组 $T[n]$ ，使 $T[i]$ 存放结点 i 的父亲；然后再写一个判别结点 U 是否为结点 V 的后代的算法。
- 假设以双亲表示法作树的存储结构，写出双亲表示的类型说明，并编写求给定的树的深度的算法。(注:已知树中结点数)
- 设计这样的二叉树, 用它可以表示父子、夫妻和兄弟三种关系, 并编写一个查找任意父亲结点的所有儿子的过程。
- 在一棵以二叉链表表示的二叉树上，试写出用按层次顺序遍历二叉树的方法，统计树中具有度为 1 的结点数目的算法。二叉链表的类型定义为：

```

typedef struct BiTNode
{
    TelemType data;
    struct BiTNode *lchild,*rchild;
} BiTNode,*BiTree;

```

- 设一棵二叉树以二叉链表为存贮结构，结点结构为(lchild, data,rchild)，设计一个算法将二叉树中所有结点的左，右子树相互交换。
- 设一棵二叉树的根结点指针为 T ， C 为计数变量，初值为 0，试写出对此二叉树中结点计数的算法：BTLC(T,C)。
- 编写程序段，利用中序全线索树求其中任意结点 p 的前序后继结点，结果仍用 p 指出。要求先描述结构和算法思路。设线索树不带头结点，其中序序列第一结点的左标志和最后结点的右标志皆为 0 (非线索)，对应指针皆为空。

参考答案：

一、选择题

1. D	2. D	3. D	4. A	5. B	6. E	7. D	8. C	9. C	10. B	11. D
12. A	13. A	14. C	15. C	16. C	17. B	18. C	19. D	20. B	21. B	22. C
23. D	24. B	25. C	26. A	27. C	28. C	29. C	30. D	31. B	32. D	

二、判断题

1. ×	2. ×	3. ×	4. √	5. √	6. √	7. √	8. ×	9. ×	10. ×
11. ×	12. √	13. √	14. √	15. √	16. ×	17. √	18. ×	19. ×	20. √
21. ×	22. ×	23. √	24. ×	25. ×	26. √	27. ×	28. ×	29. √	30. ×
31. ×	32. ×	33. √	34. √	35. ×	36. ×	37. ×	38. √		

三、填空题

1. (1)根结点(2)左子树(3)右子树	2. $p \rightarrow lchild == \text{null} \ \&\& \ p \rightarrow rchild == \text{null}$
3. (1) $++a*b3*4-cd$ (2)18	4. 平衡因子
5. 9	6. 12
7. (1) 2^{h-1} (2) 2^h-1 (3) $H=\lfloor \log_2 N \rfloor +1$	8. $\lfloor \log_2 i \rfloor = \lfloor \log_2 j \rfloor$
9. (1)0 (2) $(n-1)/2$ (3) $(n+1)/2$ (4) $\lfloor \log_2 n \rfloor +1$	10. $N2+1$
11. $\lfloor N/2 \rfloor$	12. 2^{k-2}
13. 99	14. 69
15. (1)2 (2) $n-1$ (3)1 (4) n (5)1 (6) $n-1$	16. (1)a (2)dbec (3)hfcg
17. (1).D.G.B.A.E.H.C.F. (2)...GD.B...HE..FCA	18. 二叉排序树
19. 二叉树	20. 前序
21.双亲的右子树中最左下的叶子结点	22. 31(x 的后继是经 x 的双亲 y 的右子树中最左下的叶结点)
23. (1)80 (2)001 (不唯一)	24. $2n_0-1$

四、应用题

1. (1) k^{h-1} (h 为层数)

(2) 因为该树每层上均有 K^{h-1} 个结点, 从根开始编号为 1, 则结点 i 的从右向左数第 2 个孩子的结点编号为 ki 。设 n 为结点 i 的子女, 则关系式 $(i-1)k+2 \leq n \leq ik+1$ 成立, 因 i 是整数, 故结点 n 的双亲 i 的编号为 $\lfloor (n-2)/k \rfloor +1$ 。

(3) 结点 n ($n>1$) 的前一结点编号为 n-1 (其最右边子女编号是 $(n-1)*k+1$), 故结点 n 的第 i 个孩子的编号是 $(n-1)*k+1+i$ 。

(4) 根据以上分析, 结点 n 有右兄弟的条件是, 它不是双亲的从右数的第一子女, 即 $(n-1)\%k \neq 0$, 其右兄弟编号是 n+1。

2. 证明 设度为 1 和 2 及叶子结点数分别为 n_0 , n_1 和 n_2 , 则二叉树结点数 n 为 $n=n_0+n_1+n_2$ (1)

再看二叉树的分支数, 除根结点外, 其余结点都有一个分支进入, 设 B 为分支总数, 则 $n=B+1$ 。度为 1 和 2 的结点各有 1 个和 2 个分支, 度为 0 的结点没有分支, 故 $n=n_1+2n_2+1$

(2) 由 (1) 和 (2), 得 $n_0=n_2+1$ 。

3. 给定二叉树结点的前序序列和对称序 (中序) 序列, 可以唯一确定该二叉树。因为前序序列的第一个元素是根结点, 该元素将二叉树中序序列分成两部分, 左边 (设 l 个元素) 表示左子树, 若左边无元素, 则说明左子树为空; 右边 (设 r 个元素) 是右子树, 若为空, 则右子树为空。根据前序遍历中“根—左子树—右子树”的顺序, 则由从第二元素开始的 l 个结点序列和中序序列根左边的 l 个结点序列构造左子树, 由前序序列最后 r 个元素序列与中序序列根右边的 r 个元素序列构造右子树。

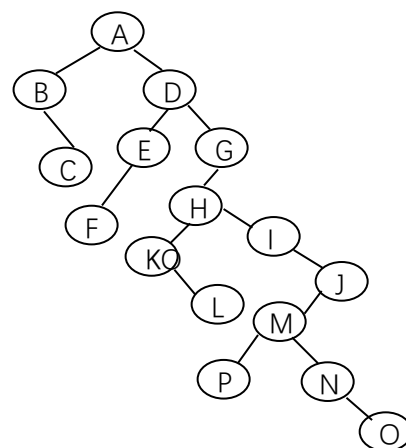
由二叉树的前序序列和后序序列不能唯一确定一棵二叉树, 因无法确定左右子树两部分。例如, 任何结点只有左子树的二叉树和任何结点只有右子树的二叉树, 其前序序列相同, 后序序列相同, 但却是两棵不同的二叉树。

4. 森林转为二叉树的三步:

- (1) 连线 (将兄弟结点相连, 各树的根看作兄弟);
- (2) 切线 (保留最左边子女为独生子女, 将其它子女分枝切掉);
- (3) 旋转 (以最左边树的根为轴, 顺时针向下旋转 45 度)。

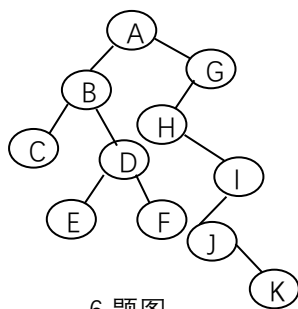
其实经过 (1) 和 (2), 已转为二叉树,

执行 (3) 只是为了与平时的二叉树的画法一致。

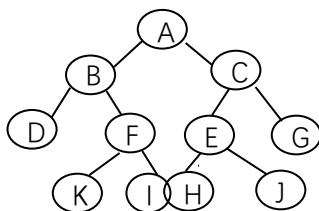


5.略

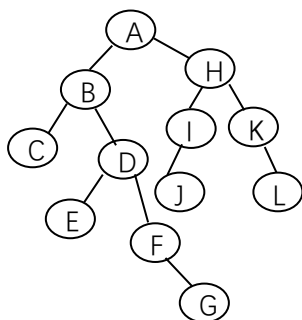
6.



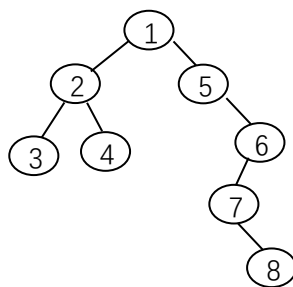
6 题图



6 (1)



6 (2)



6 (3)

7 .

Data	A	B	C	D	E	F	G	H	I	J	K
Ltag	0	1	0	0	0	1	0	1	1	1	1
Fch	2	null	5	7	8	5	11	2	8	9	3
Rtag	1	0	0	1	0	1	1	0	0	1	1
Nsib	null	3	4	1	6	3	4	9	10	5	7

8 . 首先确定是否需加“虚权值” (即权值为 0) ,对 m 个权值, 建 k 叉树, 若 $(m-1) \% (k-1) = 0$, 则不需要加“虚权值”, 否则, 第一次归并时需 $(m-1) \% (k-1) + 1$ 个权值归并。建立 k 叉树的过程如下:

- (1) 将 m 个权值看作 m 棵只有根结点的 k 叉树的集合 $F = \{T_1, T_2, \dots, T_m\}$ 。
- (2) 从 F 中选 k (若需加虚权值, 则第一次选 $(m-1) \% (k-1) + 1$ 个权值最小的树作子树, 构成一棵 k 叉树, k 叉树根结点的权值为所选的 k 个树根结点权值之和, 在 F 中删除这 k 棵子树, 并将新 k 叉树加入到 F 中。
- (3) 从 F 中选 k 个权值最小的树作子树, 构成一棵 k 叉树, 其根结点权值等于所选的 k 棵树根结点权值之和, 在 F 中删除这 k 棵树, 并将新得到的树加到 F 中。
- (4) 重复 (3), 直到 F 中只有一棵树为止, 这就是最优的 k 叉树。对本题 10 个权值, 构造最优三叉树。

因 $(10-1) \% (3-1) = 1$, 所以第一次用 2 个权值合并。

最小加权路径长度:

$$(1+4) * 4 + (9+16) * 3 + (25+36+49+64+81) * 2 + 100 * 1 = 705$$

9.

	字符	weight	Parent	lch	rch		字符	weight	parent	lch	rch
1	A	3	0	0	0	1	A	3	8	0	0
2	B	12	0	0	0	2	B	12	12	0	0
3	C	7	0	0	0	3	C	7	10	0	0
4	D	4	0	0	0	4	D	4	9	0	0
5	E	2	0	0	0	5	E	2	8	0	0
6	F	8	0	0	0	6	F	8	10	0	0
7	G	11	0	0	0	7	G	11	11	0	0
8			0	0	0	8		5	9	5	1
9			0	0	0	9		9	11	4	8
10			0	0	0	10		15	12	3	6
11			0	0	0	11		20	13	9	7
12			0	0	0	12		27	13	2	10
13			0	0	0	13		47	0	11	12

五、算法设计题

1.[题目分析]当森林（树）以孩子兄弟表示法存储时，若结点没有孩子（fch=null），则它必是叶子，总的叶子结点个数是孩子子树（fch）上的叶子数和兄弟（nsib）子树上叶结点个数之和。

```
typedef struct node
{
    ElemType data; //数据域
    struct node *fch, *nsib; //孩子与兄弟域
} *Tree;

int Leaves (Tree t)
//计算以孩子-兄弟表示法存储的森林的叶子数
{
    if(t)
    {
        if(t->fch==null) //若结点无孩子，则该结点必是叶子
            return(1+Leaves(t->nsib)); //返回叶子结点和其兄弟子树中的叶子结点数
        else return (Leaves(t->fch)+Leaves(t->nsib)); //孩子子树和兄弟子树中叶子数之和
    }
} //结束 Leaves
```

2. [题目分析]由指示结点 i 左儿子和右儿子的两个一维数组 L[i]和 R[i]，很容易建立指示结点 i 的双亲的一维数组 T[i]，根据 T 数组，判断结点 U 是否是结点 V 后代的算法，转为判断结点 V 是否是结点 U 的祖先的问题。

```
int Generation (int U,V,N,L[],R[],T[])
//L[]和 R[]是含有 N 个元素且指示二叉树结点 i 左儿子和右儿子的一维数组，
//本算法据此建立结点 i 的双亲数组 T，并判断结点 U 是否是结点 V 的后代。
{
    for(i=1;i<=N;i++) T[i]=0; //T 数组初始化
    for (i=1;i<=N;i++) //根据 L 和 R 填写 T
    {
        if(L[i]!=0) T[L[i]]=i; //若结点 i 的左子女是 L，则结点 L 的双亲是结点 i
        for(i=1;i<=N;i++)
        {
            if (R[i]!=0) T[R[i]]=i; //i 的右子女是 r，则 r 的双亲是 i
        }
        int parent=U; //判断 U 是否是 V 的后代
        while (parent!=V && parent!=0) parent=T[parent];
        if (parent==V){printf("结点 U 是结点 V 的后代");return(1);}
        else{ printf("结点 U 不是结点 V 的后代");return(0);}
    }
} //结束 Generation
```

3. [题目分析] 由于以双亲表示法作树的存储结构，找结点的双亲容易。因此我们可求出每一结点的层次，取其最

大层次就是树有深度。对每一结点，找其双亲，双亲的双亲，直至（根）结点双亲为 0 为止。

```
int Depth(Ptree t)    //求以双亲表示法为存储结构的树的深度，Ptree 的定义参看教材
{int maxdepth=0;
 for(i=1;i<=t.n;i++)
 {temp=0; f=i;
  while(f>0) {temp++; f=t.nodes[f].parent; }    // 深度加 1,并取新的双亲
  if(temp>maxdepth) maxdepth=temp; //最大深度更新}
 return(maxdepth); //返回树的深度
} //结束 Depth
```

4.[题目分析]用二叉树表示出父子，夫妻和兄弟三种关系，可以用根结点表示父（祖先），根结点的左子女表示妻，妻的右子女表示子。这种二叉树可以看成类似树的孩子兄弟链表表示法；根结点是父，根无右子女，左子女表示妻，妻的右子女（右子女的右子女等）均可看成兄弟（即父的所有儿子），兄弟结点又成为新的父，其左子女是兄弟（父的儿子）妻，妻的右子女（右子女的右子女等）又为儿子的儿子等等。首先递归查找某父亲结点，若查找成功，则其左子女是妻，妻的右子女及右子女的右子女等均为父亲的儿子。

```
BiTree Search(BiTree t,ElemType father) //在二叉树上查找值为 father 的结点
{if(t==null) return (null);           //二叉树上无 father 结点
 else if(t->data==father) return(t);   //查找成功
 p=Search(t->lchild,father); p=Search(t->rchild,father); }
} //结束 Search

void PrintSons(BiTree t,ElemType p) //在二叉树上查找结点值为 p 的所有的儿子
{p=Search(t,p);                       //在二叉树 t 上查找父结点 p
 if(p!=null)                          //存在父结点
 {q=p->lchild; q=q->rchild;             //先指向其妻结点，再找到第一个儿子
  while(q!=null) {printf(q->data); q=q->rchild;} //输出父的所有儿子
 }
} //结束 PrintSons
```

5. int Level(BiTree bt) //层次遍历二叉树，并统计度为 1 的结点的个数

```
{int num=0; //num 统计度为 1 的结点的个数
 if(bt){QueueInit(Q); QueueIn(Q,bt); //Q 是以二叉树结点指针为元素的队列
  while(!QueueEmpty(Q))
  {p=QueueOut(Q); printf(p->data);    //出队,访问结点
   if(p->lchild && !p->rchild ||!p->lchild && p->rchild) num++; //度为 1 的结点
   if(p->lchild) QueueIn(Q,p->lchild); //非空左子女入队
   if(p->rchild) QueueIn(Q,p->rchild); //非空右子女入队
  } } //if(bt)
 return(num); } //返回度为 1 的结点的个数
```

6. void exchange(BiTree bt) //将二叉树 bt 所有结点的左右子树交换

```
{if(bt){BiTree s;
 s=bt->lchild; bt->lchild=bt->rchild; bt->rchild=s; //左右子女交换
 exchange(bt->lchild); //交换左子树上所有结点的左右子树
 exchange(bt->rchild); //交换右子树上所有结点的左右子树
 } }
```

[算法讨论]将上述算法中两个递归调用语句放在前面，将交换语句放在最后，则是以后序遍历方式交换所有结点的左右子树。中序遍历不适合本题。

7. int BTLC(BiTree T,int *c) //对二叉树 T 的结点计数

```
{if(T)
 {*c++;
  BTLC(T->lchild,&c); //统计左子树结点
```

```
BTLC(T->rchild,&c); //统计右子树结点
```

```
} } //结束 Count,调用时*c=0
```

8. [题目分析]求中序全线索树任意结点 p 的前序后继，其规则如下：若 p 有左子女，则左子女就是其前序后继；若 p 无左子女而有右子女，则 p 的右子女就是 p 的前序后继；若 p 无左右子女，这时沿 p 的右线索往上，直到 p 的右标志为 0（非线索），这时若 p 的右子女为空，则表示这是中序遍历最后一个结点，故指定结点无前序后继，否则，该结点就是指定结点的前序后继。程序段如下：

```
if(p->ltag==0 && p->lchild!=null) return(p->lchild); //p 的左子女是 p 的前序后继
```

```
else if(p->rtag==0) && p->rchild!=null) return(p->rchild); //p 右子女是其前序后继
```

```
else //p 无左右子女，应沿右线索向上（找其前序后继），直到某结点右标记为 0
```

```
{while (p->rtag==1) p=p->rchild;
```

```
if (p->rchild) return(p->rchild);else return(null); } //指定结点的前序后继
```

[算法讨论]请注意题目“中序序列第一结点的左标志和最后结点的右标志皆为 0（非线索），对应指针皆为空”的说明。若无这一说明，只要结点的左标记为 0，其左子女就是其前序后继。最后，当 p 无子女，沿右线索向上找其前序后继时，若最后结点的右标志为 0，但对应指针为空， p 也无前序后继。